Sviluppo di un sistema embedded per il controllo della temperatura in camera di collaudo

Tutor Universitario: Prof. Carlo Dossi

Tutor Aziendale: Edoardo Scaglia

Candidato: Mattia Papaccioli - 747053

AMEL s.r.l. x Università degli Studi dell'Insubria

Discussione di Laurea Triennale

Il contesto

- Camera di collaudo presso AMEL s.r.l.
- Test su carichi resistivi
- Produzione di calore significativa
- Necessità di raffreddamento controllato
- Sistema precedente: manuale e impreciso

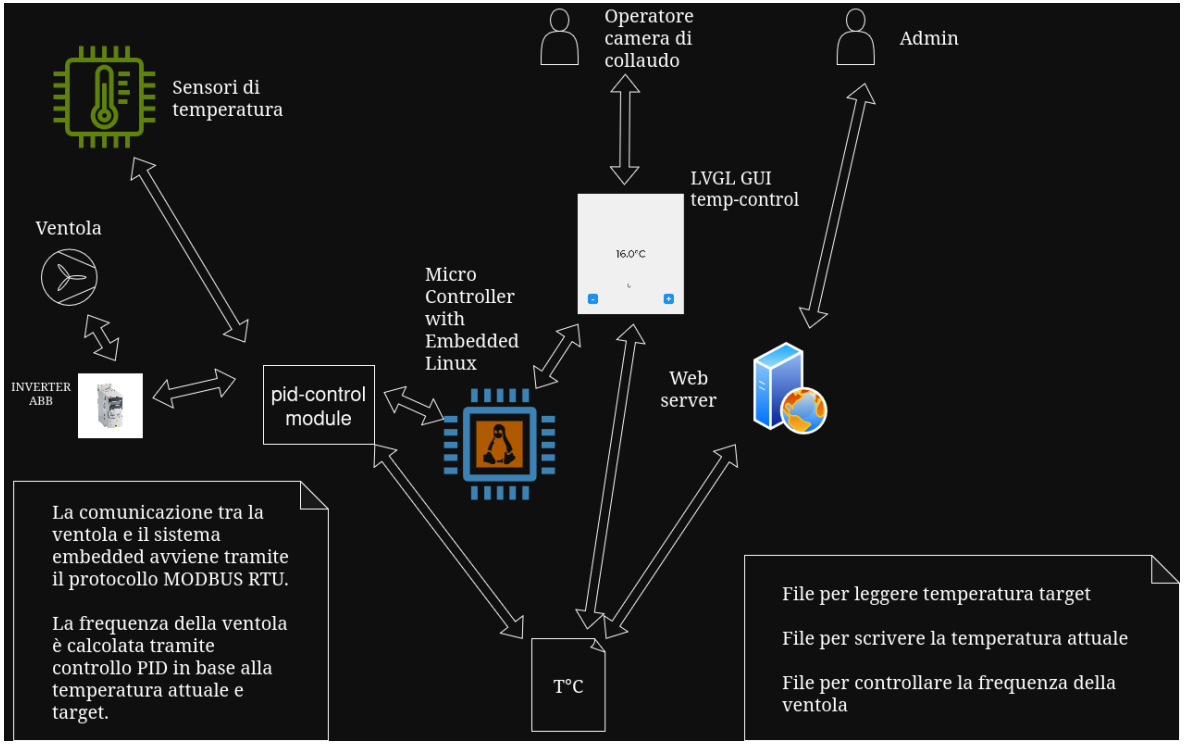
Obiettivo:

Realizzare un sistema automatico di controllo temperatura con interfaccia user-friendly

La soluzione proposta

Componenti:

- **Sensori:** DS18B20 su bus 1-Wire
- **Controllo:** Algoritmo PID
- **Attuatore:** Inverter + ventola



Hardware utilizzato

Host-board Ganador:

- Memoria SD
- Interfaccia Ethernet
- Display touchscreen
- Porta seriale RS-232

System-on-module Vulcano-A5:

- CPU ARM9 @ 400MHz
- 128 MB DDR2 SDRAM
- 256 MB NAND Flash
- Controller LCD

Periferiche:

- 2 sensori DS18B20
- Inverter ACS310
- Ventola di raffreddamento
- Adattatore USB-Seriale

Architettura del sistema

temp-control	common-control	pid-control
Interfaccia LVGL	Logging e utility	Controllo PID
Touchscreen	Configurazione	Sensori 1-Wire
GUI utente	Script init/run	Comunicazione Modbus

Comunicazione inter-processo:

- File condivisi in `/opt/amel/`
- Segnali UNIX (`SIGUSR1`)
- Temperatura target e corrente

Controllo PID

Componenti:

- **P** (Proporzionale): reazione all'errore attuale
- **I** (Integrale): accumulo errore nel tempo
- Setpoint: temperatura target

Implementazione:

- Timer monotonico per campionamento regolare
- Massima priorità scheduler
- Logging CSV per analisi

Comunicazione:

Protocollo Modbus RTU

Controllo inverter via RS-232

Regolazione frequenza ventola

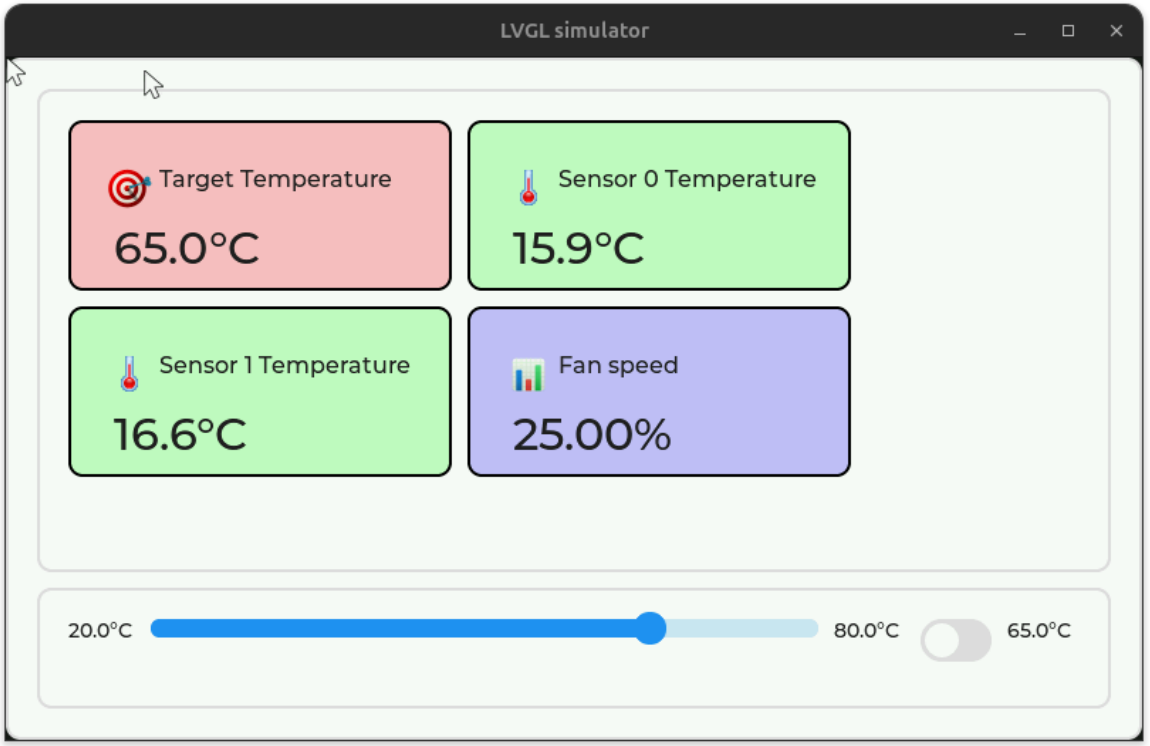
Interfaccia utente LVGL

Caratteristiche:

- Display touchscreen
- Temperatura target regolabile
- Lettura sensori in tempo reale
- Backend: libevdev + framebuffer

Sviluppo:

- Cross-compilazione ARM
- Branch separate (dev/target)
- CMake toolchain



Sistema Embedded Linux

Buildroot:

- Kernel Linux personalizzato
- Solo moduli essenziali
- Rootfs ottimizzato
- Cross-compilazione automatica

Boot sequence:

AT91bootstrap → Barebox → Linux

Pacchetti custom:

- `amel-common-control`
- `amel-temp-control`
- `amel-pid-control`

Servizi:

- SSH server
- Web server CGI
- NTP client

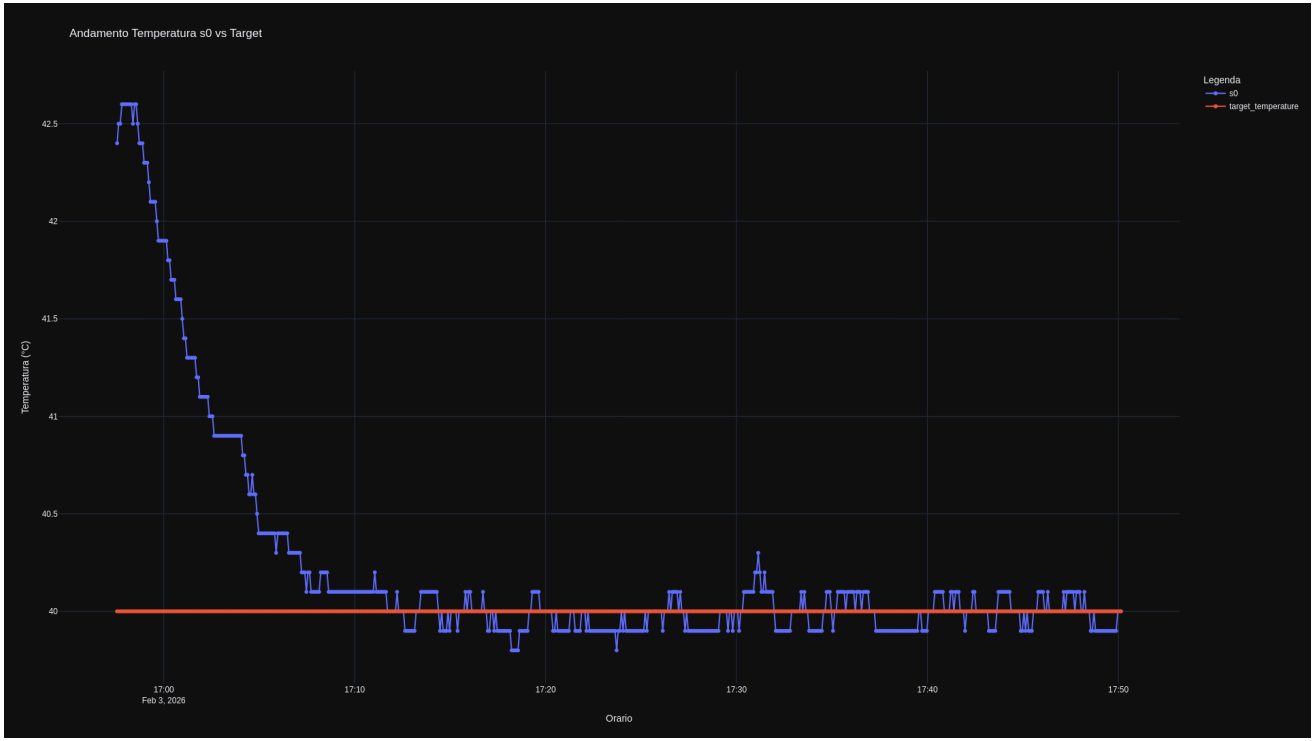
Installazione e tuning

Configurazione inverter:

- Abilitazione controllo remoto
- Comando start: 0x047f
- Registri Modbus

Tuning PID:

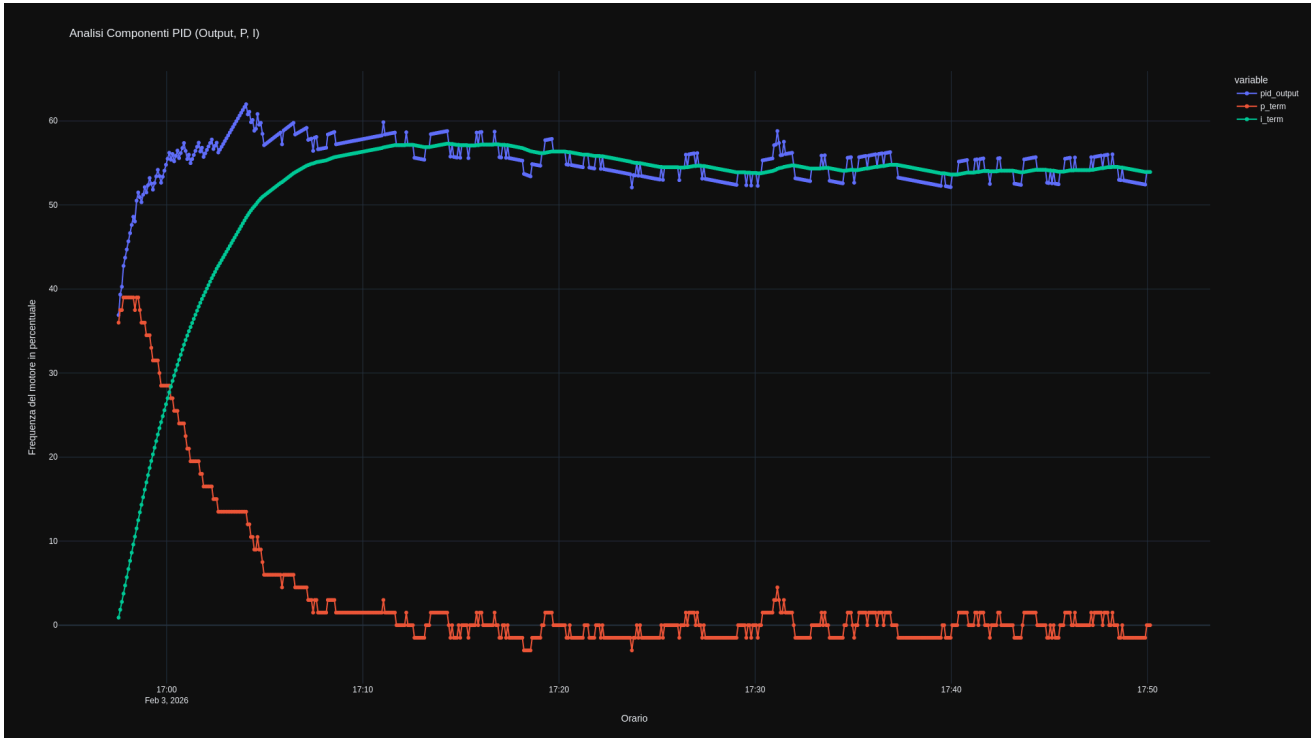
- Approccio empirico
- Test a 50°C
- Parametro P: −3000
- Parametro I: −15



Risultati e bug risolti

Problemi risolti:

- LVGL single-threaded ⟶ uso di timer
- File descriptor esauriti ⟶ connessione Modbus persistente
- Calibrazione touchscreen



Conclusioni

Risultati raggiunti:

- Sistema funzionante in produzione
- Controllo automatico temperatura
- Interfaccia user-friendly
- Accesso remoto via web

Lesson learned:

- Importanza testing sul campo
- Gestione risorse embedded
- Protocolli industriali

Sviluppi futuri:

- Dashboard web avanzata
- Logging su database
- Notifiche alert

Ringraziamenti

AMEL s.r.l. e Edoardo per il supporto tecnico

Università degli Studi dell'Insubria

e il professor Dossi

Famiglia e amici

Grazie per l'attenzione!